# JavaCPUtils: Pair Family API Cheat Sheet

**Unified API across `Pair<F, S>`, `IntPair`, and `LongPair`.**

*Rule: Any method shown below works identically for `Pair`, `IntPair`, and `LongPair`.*

---

## 1. Quick Class Selector

| Class | Field Types | Memory | Primary Use Case |
|---|---|---|---|
| **`Pair<F, S>`** | Generic Objects (`F`, `S`) | ~64 bytes | Mixed types (`Pair<String, Integer>`), floating point, custom objects, nested pairs. |
| **`IntPair`** | Primitive `int first, second;` | **24 bytes** | **3x faster**, 60% less RAM. Coordinate compression, BFS, standard CP graph edges. |
| **`LongPair`** | Primitive `long first, second;` | **24 bytes** | Dijkstra shortest path, MST with edge weights up to $10^{18}$, large coordinate grids. |

---

## 2. Construction & Direct Public Fields

```
// Option A: Using constructor
IntPair p = new IntPair(10, 20);

// Option B: Using static factory (zero diamond operator)
var p = IntPair.of(10, 20);
var gp = Pair.of("Alice", 100);

// Direct C++ style read and write:
p.first += 5;           // p.first is now 15
p.second = 99;          // p.second is now 99
System.out.println(p.first + " " + p.second);
```

---

## 3. Complete Methods Reference Table

| Operation | Method Signature | Description & Example |
|---|---|---|
| **Field Access** | `p.first`, `p.second` | Direct public fields. Zero getter/setter boilerplate. |

| | | |
|---|---|---|
| **Batch Update** | `p.set(a, b)` | Updates both fields simultaneously in-place: `p.set(5, 10);` |
| **Swap Elements** | `p.swap()` | Returns a new pair with reversed values: `IntPair rev = p.swap();` |
| **Natural Compare** | `p1.compareTo(p2)` | Lexicographical order (< 0, == 0, > 0). Compares `first`; tie-breaks on `second`. |
| **Sort by First** | `IntPair.comparingFirst()` | Built-in `Comparator` sorting by `first` ascending. |
| **Sort by Second** | `IntPair.comparingSecond()` | Built-in `Comparator` sorting by `second` ascending. |
| **Equality** | `p1.equals(p2)` | Checks value equality. Handles `null` safely. |
| **Hash Distribution** | `p.hashCode()` | Generates integer hash. `IntPair` uses direct arithmetic (`31 * first + second`). |
| **String Representation** | `p.toString()` | Formatted representation: returns `"(10, 20)"`. |
| **Duplication** | `p.clone()` | Returns an independent copy of the pair object. |

---

## 4. Sorting, PriorityQueue & Collection Examples

### A. Automatic Lexicographical Sorting

```
List<IntPair> list = new ArrayList<>();
list.add(new IntPair(5, 20));
list.add(new IntPair(2, 50));
list.add(new IntPair(5, 10));

Collections.sort(list);
// Result: (2, 50), (5, 10), (5, 20)  <- (5, 10) comes before (5, 20)
```

### B. Custom Sorting (By Second / Reversed)

```
// 1. Sort by second ascending:
list.sort(IntPair.comparingSecond());
```

```
// 2. Sort by second descending:
list.sort(IntPair.comparingSecond().reversed());

// 3. Custom lambda (e.g. by first ASC, then second DESC):
list.sort((a, b) -> {
    int cmp = Integer.compare(a.first, b.first);
    return (cmp != 0) ? cmp : Integer.compare(b.second, a.second);
});
```

### C. PriorityQueue (Min-Heap / Dijkstra)

```
// Automatically orders by smallest first, then smallest second!
PriorityQueue<LongPair> pq = new PriorityQueue<>();
pq.add(new LongPair(dist[v], v)); // (distance, node)

LongPair closest = pq.poll();     // Pops smallest distance instantly
```

### D. HashSet / HashMap

```
Set<IntPair> visited = new HashSet<>();
visited.add(new IntPair(0, 0));

if (visited.contains(new IntPair(0, 0))) {
    // Correctly returns true!
}
```